

# **Nios<sup>®</sup> V/m Processor DMA OCM Design**

## **Agilex<sup>™</sup> 5 FPGA**

## Contents

|                                     |   |
|-------------------------------------|---|
| 1. Theory of Operation .....        | 3 |
| 1.1. IP Cores .....                 | 3 |
| 2. Directory Structure .....        | 4 |
| 3. Quick Start Guide .....          | 4 |
| 4. Generating the Design .....      | 5 |
| 4.1. Building Hardware Design ..... | 5 |
| 4.2. Building Software Design ..... | 5 |
| 5. Simulating the Design .....      | 6 |
| 6. Programming the Design .....     | 6 |
| 7. Expected Results .....           | 7 |

## 1. Theory of Operation

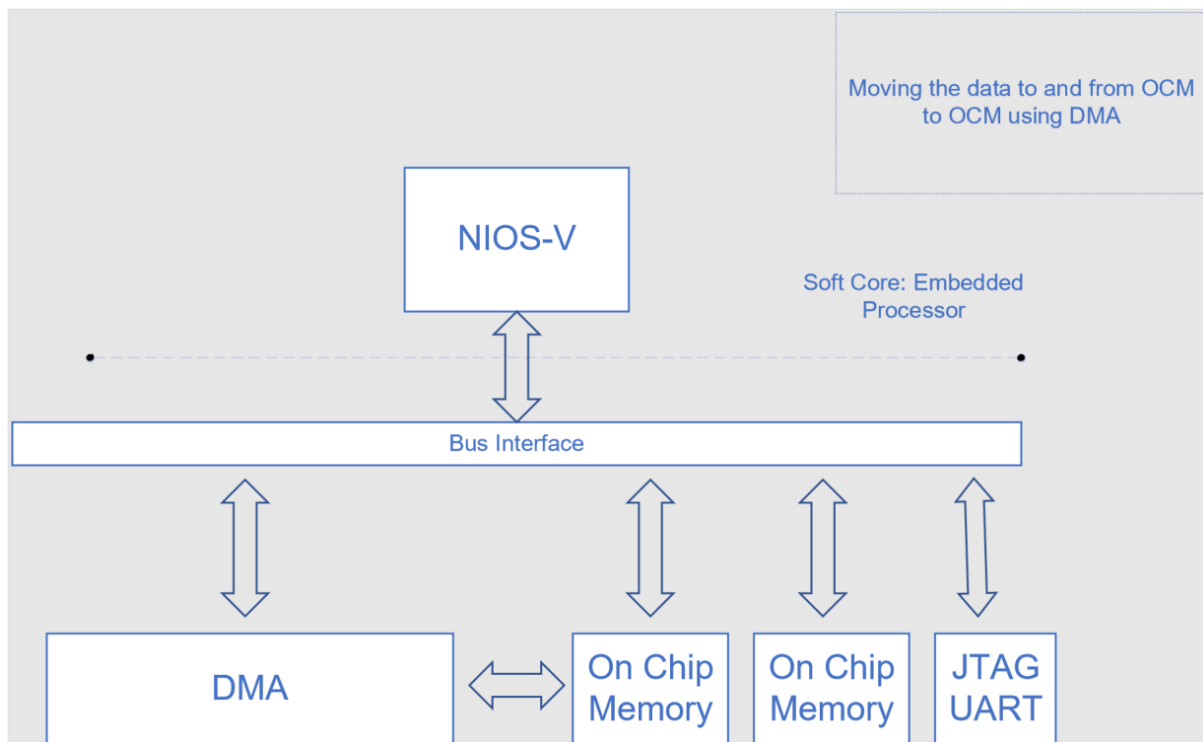
This design demonstrates the transaction between the Nios® V processor with DMA and OCM core for the Agilex™ 5 FPGA E-Series 065B Premium Development Kit (ES1) DK-A5E065BB32AES1.

This example design includes a Nios® V processor connected to the DMA, On Chip RAM-II and JTAG UART IP.

The objective of the design is to accomplish a data transfer between the 2 On Chip RAM-II using a DMA (MSGDMA) IP.

DMA facilitates the data transfer which is then read back by the processor.

**Figure 1-1. Block Diagram**



### 1.1. IP Cores

The following IPs are used in this design.

- Nios V/m soft processor core
- On Chip RAM-II
- JTAG UART
- MSGDMA
- System ID

## 2. Directory Structure

The directory structure is explained below:

Root directory: `niosv_m/niosv_m_dma_ocm`

| Directories                                                                                               | Content                                                                                                                                                                                                                                                                                                            |
|-----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| docs                                                                                                      | Document capturing the details of the example design                                                                                                                                                                                                                                                               |
| img                                                                                                       | Block diagram for the example designs                                                                                                                                                                                                                                                                              |
| ready_to_test                                                                                             | Prebuilt binaries (FPGA <code>.sof</code> file and Application <code>.elf</code> file) which can be programmed on the board and tested                                                                                                                                                                             |
| <div>sources<ul style="list-style-type: none"><li>- hw</li><li>- scripts</li><li>- sw/app</li></ul></div> | <div>Files needed to create the design<ul style="list-style-type: none"><li>- Necessary hardware files (<code>.sv</code>, <code>.v</code>, <code>.ip</code>) of the design</li><li>- This folder consists of scripts to build the design</li><li>- This folder contains software application files</li></ul></div> |

## 3. Quick Start Guide

You may try the example on the target development kit using the prebuilt binaries. The respective FPGA `sof` and Application `elf` files are found in `ready_to_test` folder.

Refer to [Chapter 6 Programming the Design](#) for the steps to program these files.

## 4. Generating the Design

The implementation of Nios® V processor system requires a hardware design developed using the Quartus® Prime software. After that, it is required to develop the software design that complements the processor system.

### 4.1. Building Hardware Design

Begin designing the system hardware by instantiating the Nios® V processor and its peripherals into Platform Designer. After configuring the system assignments and constraints, complete the hardware design by performing a successful compilation.

We will be using `sources/scripts/build_sof.py` to develop the hardware design.

This script will create a new platform design, add the IPs into the design, makes the connections, compiles the design and generates the `.sof` file.

Steps to execute the script are as follows:

1. Invoke the Nios V Command Shell in the terminal
2. Run the following command in the terminal from top level project directory:  
`> quartus_py ./scripts/build_sof.py`
3. The Quartus tool will compile the design and generate the output files into `<Root Directory>/sources/hw/output_files`

### 4.2. Building Software Design

After the processor system is ready, you may begin building the software design. It consists of the following steps:

1. Create a board support package (BSP) project.
2. Create a Nios® V processor application project with source code.
3. Build the application.
4. Convert the application `.elf` file into `.hex` file for memory initialization.

To create software application, run the following commands in the terminal:

```
> niosv-bsp -c --quartus-project=hw/top.qpf --qsys=hw/qsys_top.qsys --  
type=hal sw/bsp/settings.bsp  
> niosv-app --bsp-dir=sw/bsp --app-dir=sw/app --srcs=sw/app/main.c  
> niosv-shell  
> cmake -S ./sw/app -B sw/app/build -G "Unix Makefiles"  
> make -C sw/app/build  
> elf2hex sw/app/build/app.elf -b 0x0 -w 32 -e 0x9ffff  
sw/app/build/onchip_mem.hex -r4
```

**Note:** It is recommended to clean the `app/build` project before regenerating `elf` (`cmake` and `make`).

## 5. Simulating the Design

You may carry on simulation for this design to evaluate its intended functions. During simulation, the action of downloading .elf file is unable to be simulated. Thus, the processor memory is instead initialized with the application .hex file.

Use the following commands to run the simulation:

```
# Generate Testbench from Platform Designer.
# Generate -> Generate Testbench System

> cp ./sw/app/build/onchip_mem.hex ./qsys_top_tb/qsys_top_tb/sim/mentor
> cd hw/qsys_top_tb/qsys_top_tb/sim/mentor/
> vsim &
> source msim_setup.tcl
> ld_debug
> run -all
```

## 6. Programming the Design

Use the Quartus® Prime Programmer tool and the Ashling\* RiscFree\* IDE to program the Nios® V processor-based system into the FPGA and to run your application.

Program the generated sof, then download the elf file on the board according to the command below.

```
> quartus_pgm --cable=1 -m jtag -o 'p;top.sof'

#----- Download the elf file on the board ----#
> niosv-download -g app.elf -c 1

#----- Verify the output on the terminal by using the following command in the
terminal ----#
> juart-terminal -d 1 -c 1 -i 0
```

**Note:** Reduce the JTAG clock frequency to 6MHz before programming the application .elf file on the board using the command: `jtagconfig --setparam 1 JtagClock 6M`

## 7. Expected Results

The following is the output as observed on the JTAG UART terminal. The output is analogous to the logic from the application code. Users should be able to observe the same output on their terminal/setup.

Figure 1-2. Expected Result

```
# Hello World from NIOSV/m core on SM !
# *****Run DMA Test - OCM to OCM *****
# writing into onchip memory from NIOS V
# Addr 0x200000 Data written 0x0
# Addr 0x200004 Data written 0x4
# Addr 0x200008 Data written 0x8
# Addr 0x20000c Data written 0xc
# Addr 0x200010 Data written 0x10
# Addr 0x200014 Data written 0x14
# Addr 0x200018 Data written 0x18
# Addr 0x20001c Data written 0x1c
# Data write to onchip memory completed
# DMA is open mSG-DMA
# Created descriptor definition for mSG-DMA
# status is 0
# Addr 0x120000 Data read 0x0
# base 0x120000 offset_add 0x0 ,expected 0x0 read 0x0
# Data matched !
# Addr 0x120004 Data read 0x4
# base 0x120000 offset_add 0x4 ,expected 0x4 read 0x4
# Data matched !
# Addr 0x120008 Data read 0x8
# base 0x120000 offset_add 0x8 ,expected 0x8 read 0x8
# Data matched !
# Addr 0x12000c Data read 0xc
# base 0x120000 offset_add 0xc ,expected 0xc read 0xc
# Data matched !
# Addr 0x120010 Data read 0x10
# base 0x120000 offset_add 0x10 ,expected 0x10 read 0x10
# Data matched !
# Addr 0x120014 Data read 0x14
# base 0x120000 offset_add 0x14 ,expected 0x14 read 0x14
# Data matched !
# Addr 0x120018 Data read 0x18
# base 0x120000 offset_add 0x18 ,expected 0x18 read 0x18
# Data matched !
# Addr 0x12001c Data read 0x1c
# base 0x120000 offset_add 0x1c ,expected 0x1c read 0x1c
# Data matched !
# Memory copy over DMA was a SUCCESS. Session will now close.
# Print the value of System ID
# System ID from Peripheral core is 0xA5
```